09 Blocking I/O

- I/O interface

Blocking : I/O가 완료될 때까지 대기 (sleep, block)

Non Blocking : 당장 완료할 수 없다면 실패 반환

Asynchronous : 운영체제가 알아서 백그라운드에서 완료 후 결과 통보

Blocking Outputs

write 의 반환 성공이 실제 물리 디스크에 기록된 것을 의미 X

단지 write 에 사용한 버퍼를 안전하게 재사용 가능.

페이지 캐시에만 복사되고, 실제 디스크에는 백그라운드에서 수행됨

네트워크에서의 write도 동일.

실제로 도착했음을 의미 X 단지 소켓버퍼에 복사된 것

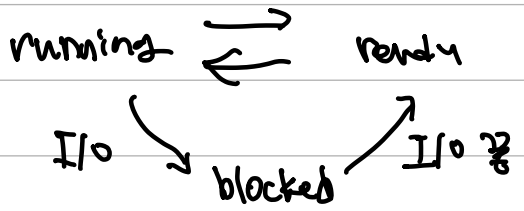
read 에서는 페이지 캐시 나 소켓버퍼에 있다면 데이터를 복사해 리턴.

없다면 가져올 때까지 sleep.

리턴 되면 읽어온 것이 버퍼에 있다는 의미.

- wait queue

특정 조건이 만족되기를 기다리는 프로세스를 저장



wait_queue_head_t 구조체

spin lock과 list_head 사용

DECLARE_WAIT_QUEUE_HEAD() 매크로 or init_waitqueue_head() 사용

- Sleep wait queue

wait_event (wq, condition)

wait_event_interruptible (wq, condition)

wait_event_interruptible_timeout (wq, condition, timeout)

현재 wait_event를 인터럽트 시그널 받아도 안깨지고 I/O 계속 기다림.

- wake up wait queue

wake_up(&wq)

wake_up_interruptible(&wq) ← 인터럽터블로 재운거 깨운다

인터럽에서 호출해 주어야 함.

- Exclusive

_exclusive 안붙으면 non 배타적인 프로세스 임.

non 배타적이면 해당 wait queue 깨울때 다 깨워남.

배타적이려면 wait_event_interruptible_exclusive() 로 재우기

non배타적인건 wait queue head 쪽에 추가, 배타적인건 tail에 추가.

wait queue 깨우면 non배타적 다 깨우고, 처음 만난 배타적 하나까지만 깨움.